Project Report: Momo Paradise

Database Design and Implementation for a Multi-Branch Shabu Buffet Chain

Chaiyanun Sakulsaowapakkul 6681299

Sorawich Pimsen 6681297

Nathanon Chirattithphan 6780756

ICCS 225 Database Foundations — Term 2025-26 T3

Mahidol University International College

Table of Contents

1. Introduction

This report details the design and implementation of a database-backed web platform for Momo Paradise, a multi-branch all-you-can-eat shabu/sukiyaki restaurant chain. The restaurant sells a time-limited buffet priced per head, with different buffet tiers unlocking premium dishes. Every branch must coordinate walk-in queues, timed reservations, roughly 90-minute dining sessions, tier-restricted ordering, billing with promotions, and a membership points program — all in real time.

The platform replaces error-prone manual coordination with a single PostgreSQL database that enforces every business rule, fronted by a thin Flask API and a React web application. The guiding design principle is that the application never touches tables directly: every read and write goes through a PostgreSQL function, so business rules are defined and enforced in exactly one place — the database.

2. Problem Statement (Pain Points)

Running an all-you-can-eat chain by hand (or with disconnected spreadsheets) creates constant operational friction. The platform directly addresses the following pain points:

- **Queues and bookings compete for the same tables.** Walk-in queues and timed reservations must be coordinated per branch in real time; doing this manually causes long waits, double-seated tables, and skipped parties.
- **Timed sessions are hard to track.** Every table is a time-limited session with a start time, an ordering window, and an end time. Without a per-table timer, staff miss overtime sessions and turn tables too slowly.
- **Tiered ordering is error-prone.** A Standard-tier table must not be able to order Wagyu. Enforcing per-dish tier rules from memory leads to wrong-tier orders and disputes at the bill.
- **Billing is complicated.** The final bill is per-head price × guest count, plus à-la-carte extras, minus coupon discounts — then membership points must be awarded. Manual computation causes billing mistakes.
- **Membership points get lost.** Points tracked on paper or in a separate system drift out of sync with actual visits, frustrating loyal customers.

3. Solution: Platform Features

The platform serves two user groups through one database. Customers use the public web app; staff use a back-office view of the same system.

Customer features (web app)

- Register and log in with a member account
- Browse branches and live table availability

- Reserve a time slot or join the walk-in queue
- Order dishes during the dining session from a tier-filtered menu
- See the running bill in real time
- View membership points balance and history

Staff features (back office)

- Seat the queue: assign waiting parties to available tables
- Open dining sessions: table + tier + guest count + timer
- Kitchen view: live feed of unserved orders, mark dishes as served
- Manage menu items and per-branch availability
- Create promotions and coupon codes
- Checkout: compute the bill, apply a discount, award points — in one transaction

Minor features of the real restaurant (gift cards, advertising, etc.) are out of scope by design, keeping the schema focused on the core operational loop of a visit.

4. Database Design

4.1 Entity-Relationship (ER) Diagram

The database consists of 14 tables covering the full lifecycle of a visit: reservation or queue entry → seating → dining session → orders → bill → membership points.

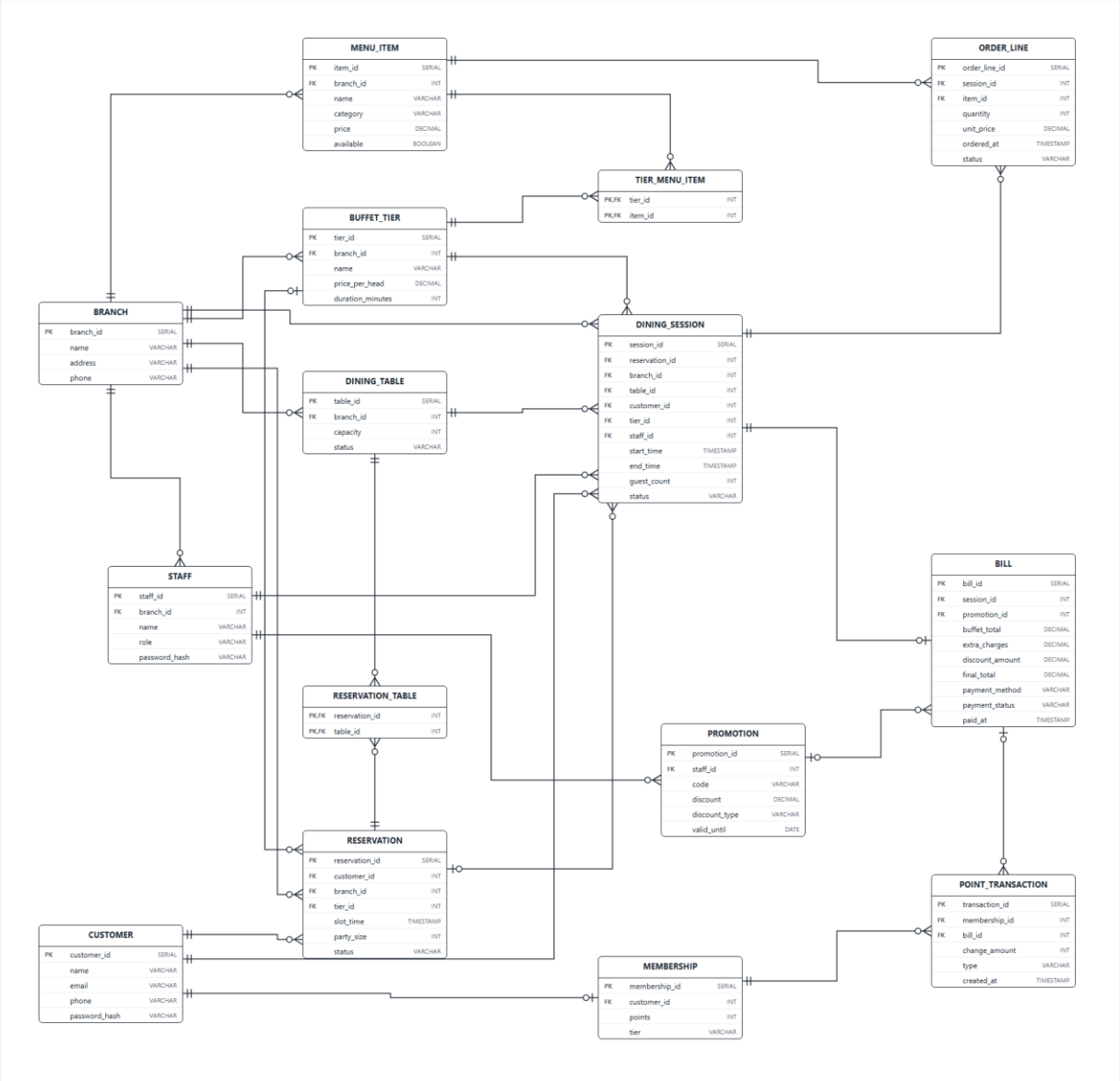


Figure 1: ER diagram of the Momo Paradise database (14 tables).

4.2 Table Details

Table	Purpose
branch	One row per restaurant location (name, address, phone).
customer	Registered customer accounts. Email is UNIQUE; passwords are stored only as bcrypt hashes.
staff	Branch employees with a role (e.g. manager); each belongs to one branch.
membership	1-to-1 with customer (enforced by a UNIQUE foreign key); holds the points balance and member tier.
dining_table	Physical tables per branch, with seating capacity (CHECK > 0) and an availability

	status.
buffet_tier	Per-branch buffet tiers: price per head and session duration in minutes.
menu_item	Dishes per branch. price = 0 means buffet-included; a positive price is an extra charge. An availability flag lets staff take dishes off the menu.
tier_menu_item	Junction table (composite PK tier_id + item_id) defining exactly which dishes each buffet tier may order — a many-to-many relationship.
reservation	One shape for both timed bookings and walk-in queue entries: slot_time NULL means a queue entry. Status is CHECK-constrained to reserved / queued / seated / cancelled / no_show.
reservation_table	Junction table (composite PK) recording which tables were assigned to a reservation at seating time — a large party may need several tables combined.
dining_session	An active timed visit: table, tier, guest count, serving staff, start/end time, status.
order_line	One dish order within a session: quantity, status, and unit_price copied from the menu at order time.
promotion	Coupon codes created by staff: percent or fixed discount (CHECK-constrained), optional expiry date.
bill	One bill per session (UNIQUE foreign key): buffet total, extra charges, discount, final total, payment method and status.
point_transaction	Membership point earns and redeems, linked to the bill that generated them.

4.3 Design Highlights

- **Tier ↔ Menu, many-to-many.** The tier_menu_item junction table defines which dishes each buffet tier may order. The signature Momo rule — a Standard table cannot order Wagyu — becomes a single EXISTS check inside fn_place_order, enforced by the database on every order.
- **One reservation shape for queue and booking.** A reservation with slot_time NULL is a walk-in queue entry; with a time it is a timed booking. One table and one status machine (reserved / queued / seated / cancelled / no_show) covers both.
- **Reservation ↔ Table, many-to-many.** Tables are not assigned at booking time. When staff seat a party, one row per assigned table is written to reservation_table — so a party of 12 can be given two 6-seat tables without bending the schema.
- **Price snapshot at order time — our one deliberate 3NF exception.** order_line.unit_price copies menu_item.price when the dish is ordered, and bill stores its computed totals. This denormalization is intentional: historical bills stay correct even if menu prices change later.

4.4 System Architecture

The stack is deliberately thin everywhere except the database:

Layer	Technology	Role
Frontend	React + TypeScript (Vite, React Router)	Customer app and staff back office; talks only to the API.

Backend	Flask (Python)	25 JSON endpoints. Its single DB helper call_fn() only executes SELECT * FROM fn_x(%s, %s) — it never writes SQL against tables.
Database	PostgreSQL (hosted on Render)	14 tables, 28 functions. All business rules live here.

Supporting tooling: Docker Compose runs the backend, frontend, and an nginx reverse proxy as one command for development and demos; deploy.sh re-runs the schema, functions, and security scripts against the Render database; DataGrip and psql were used for development, testing, and the example results in this report.

5. Application Flow and Key Screens

5.1 Customer Journey

A customer registers, picks a branch, checks availability, reserves (or joins the queue), and — once seated by staff — orders from the tier-filtered menu while watching the running bill.

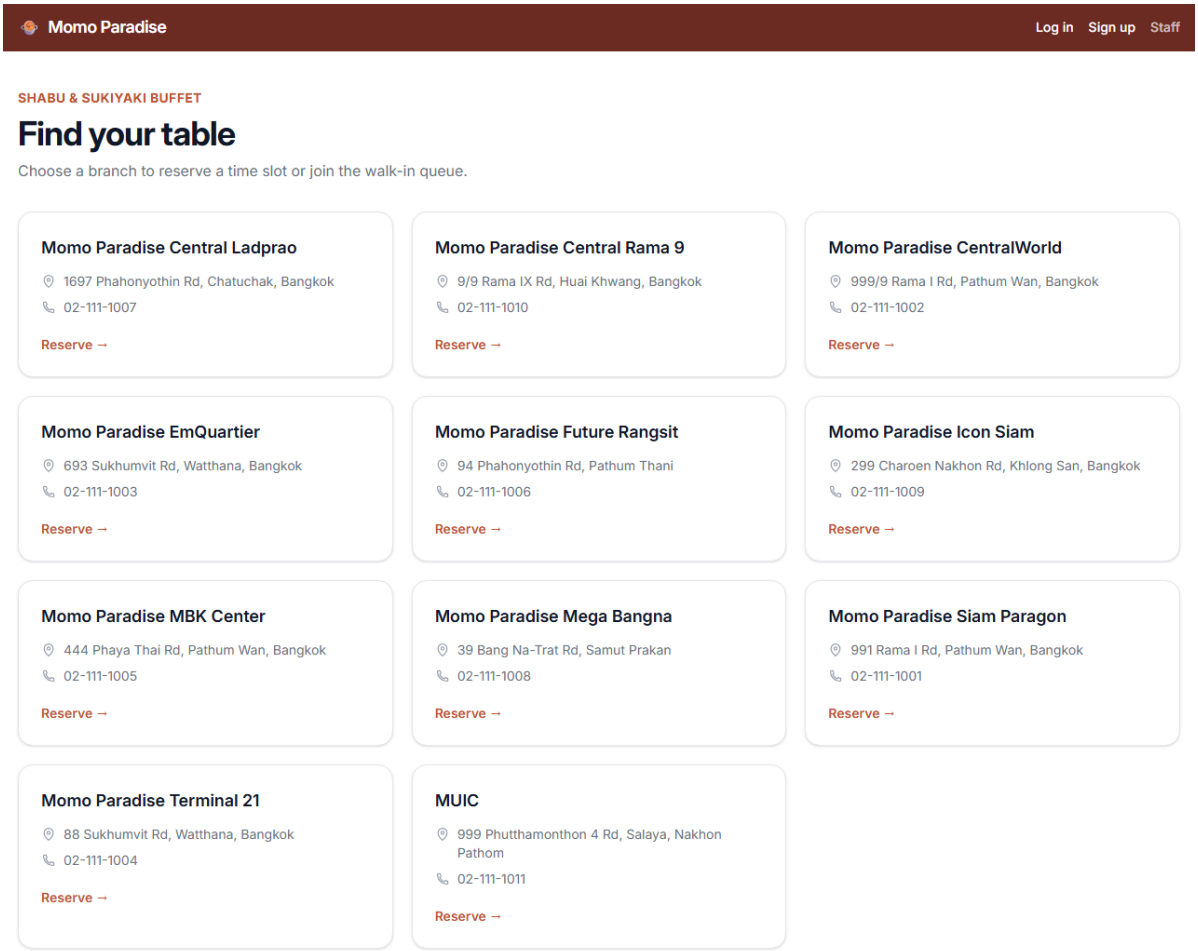


Figure 2: Home screen: the branch picker with all Momo Paradise locations — powered by fn_get_branches.

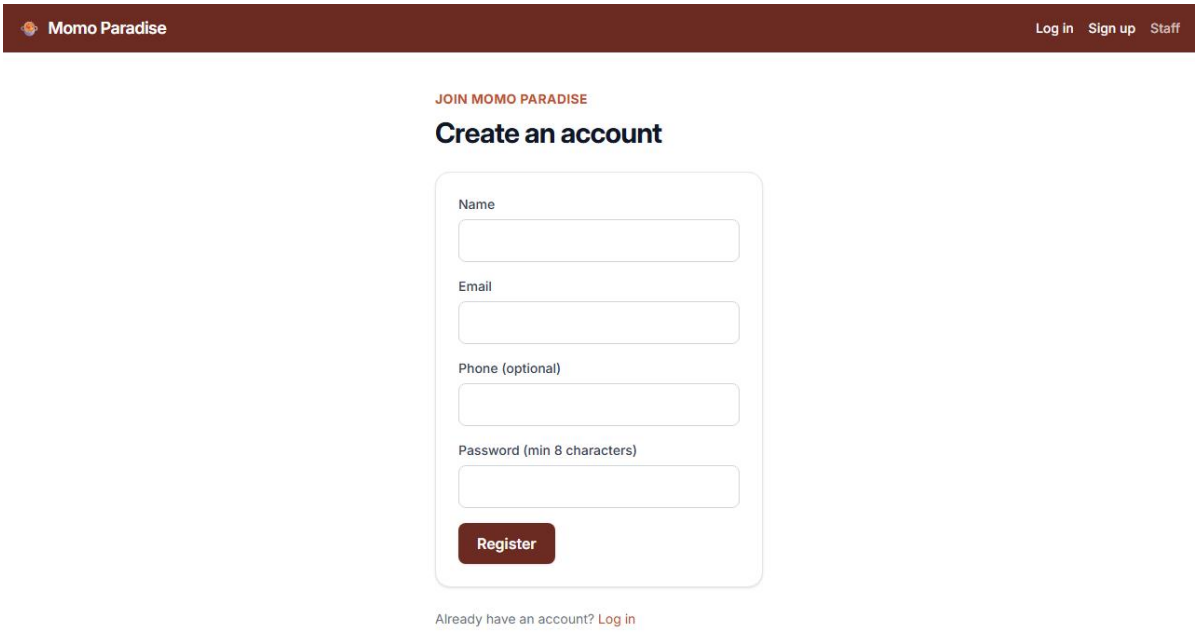


Figure 3: Account registration — fn_register_customer bcrypt-hashes the password inside the database and creates the membership row.

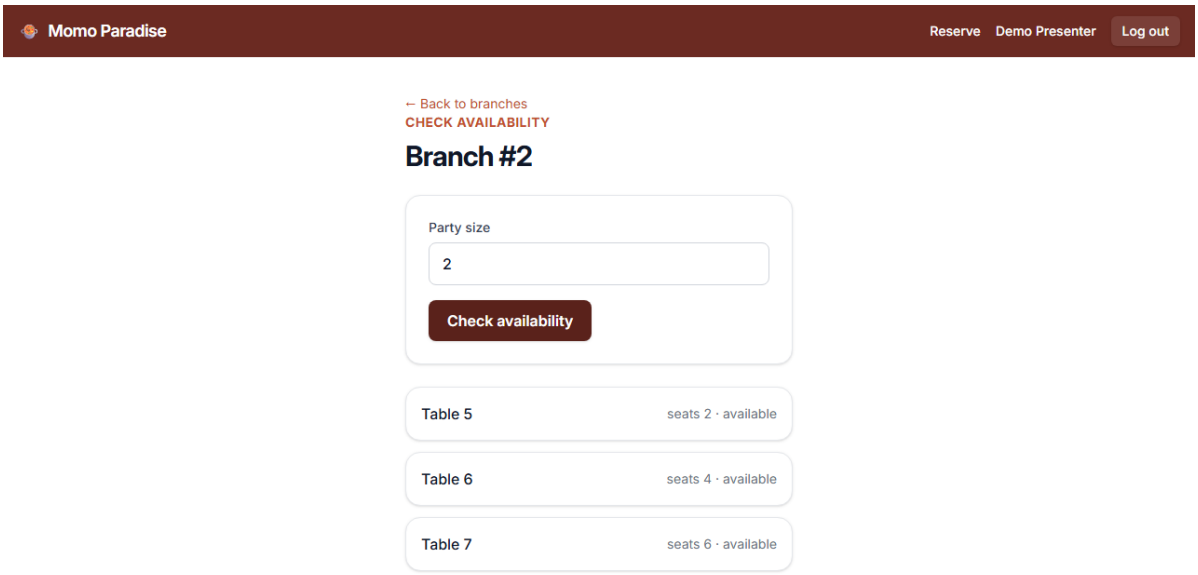


Figure 4: Branch detail: live table availability for a given party size — powered by fn_get_available_tables.

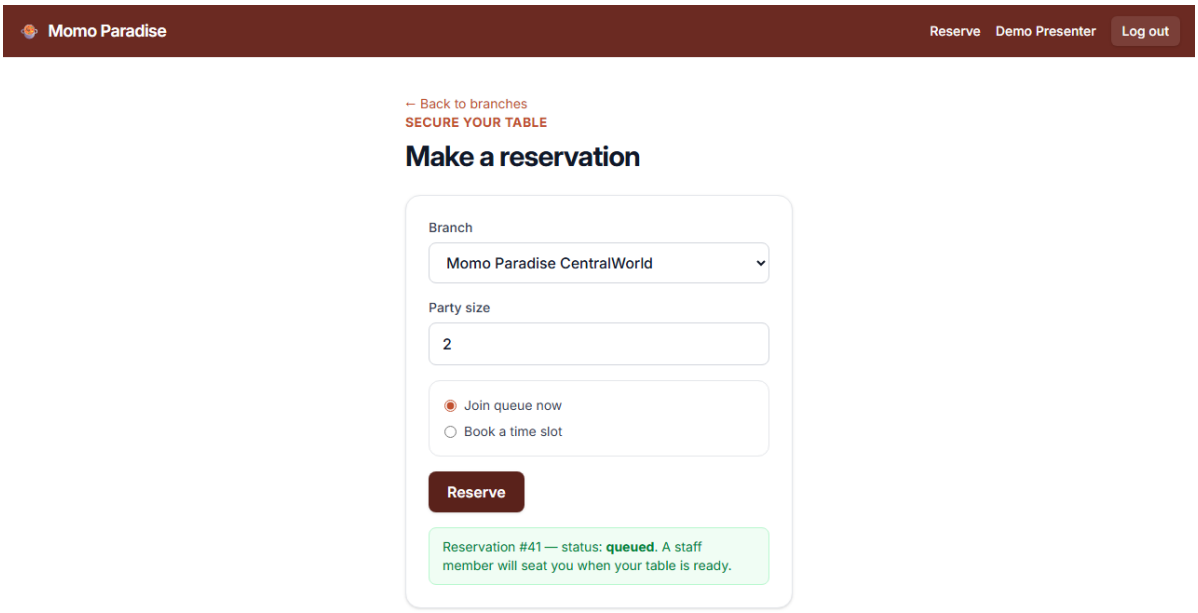


Figure 5: Making a reservation — one call, fn_create_reservation, handles both a timed booking and a walk-in queue entry.

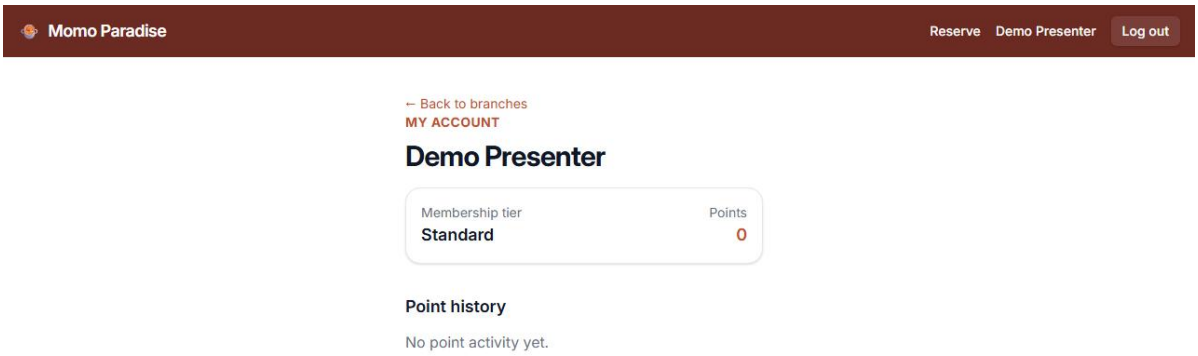


Figure 6: Member profile: membership tier, points balance, and point history — powered by fn_get_membership and fn_get_point_history.

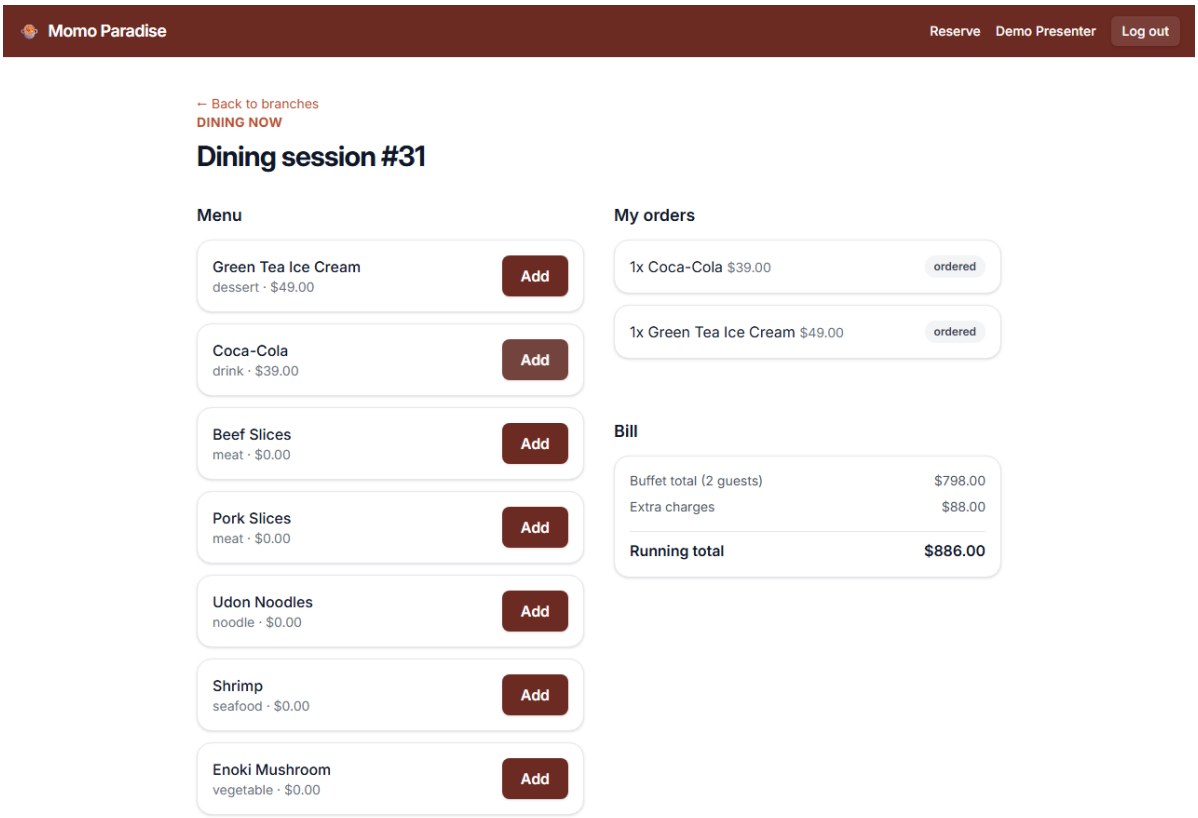


Figure 7: In-session ordering: fn_get_tier_menu filters the menu to the session's tier, fn_place_order enforces the tier rule, fn_get_session_orders lists what was ordered.

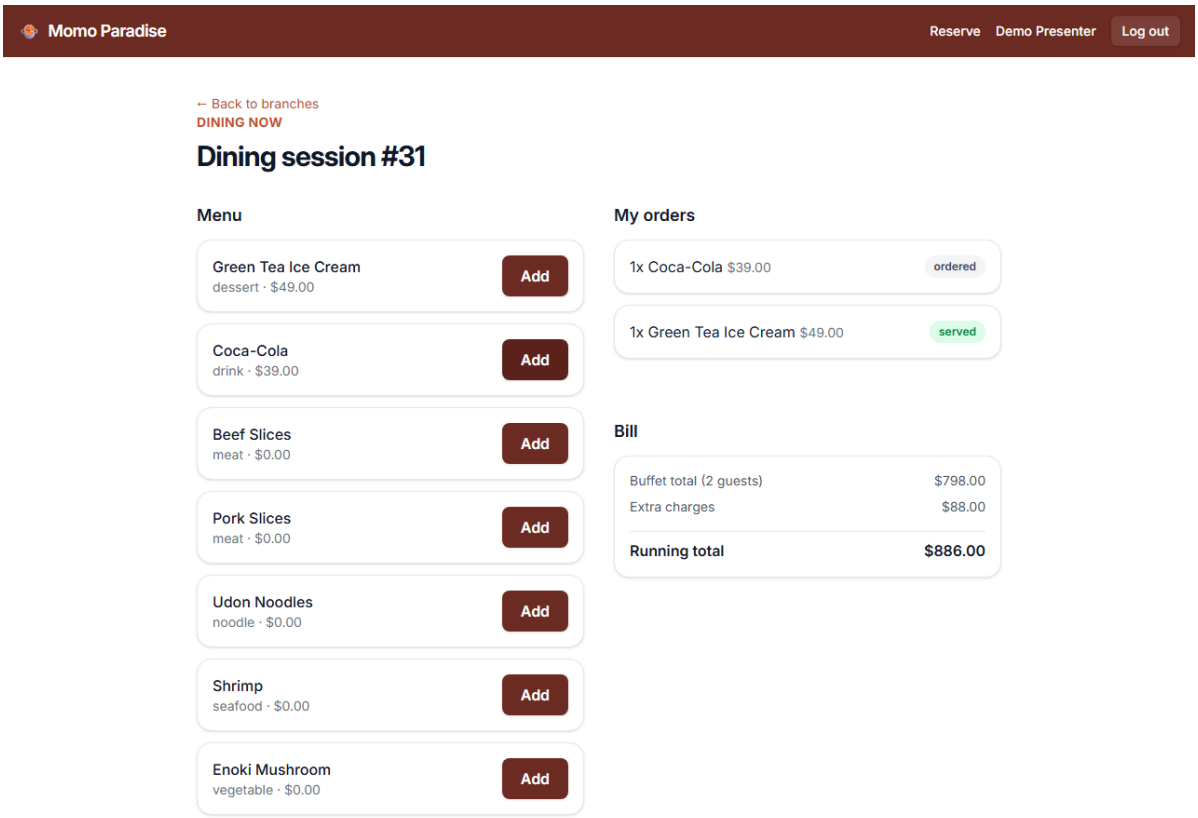


Figure 8: The running bill updates in real time — fn_get_current_bill computes tier price × guests plus extras.

5.2 Staff Back Office

Staff seat waiting parties, open timed sessions, serve orders from the kitchen feed, maintain the menu and promotions, and close each visit with a one-transaction checkout.

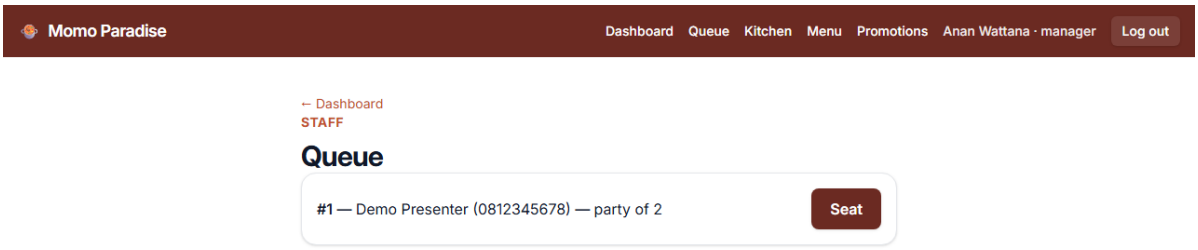


Figure 9: The queue screen: waiting parties at this branch, oldest first — powered by fn_get_queue.

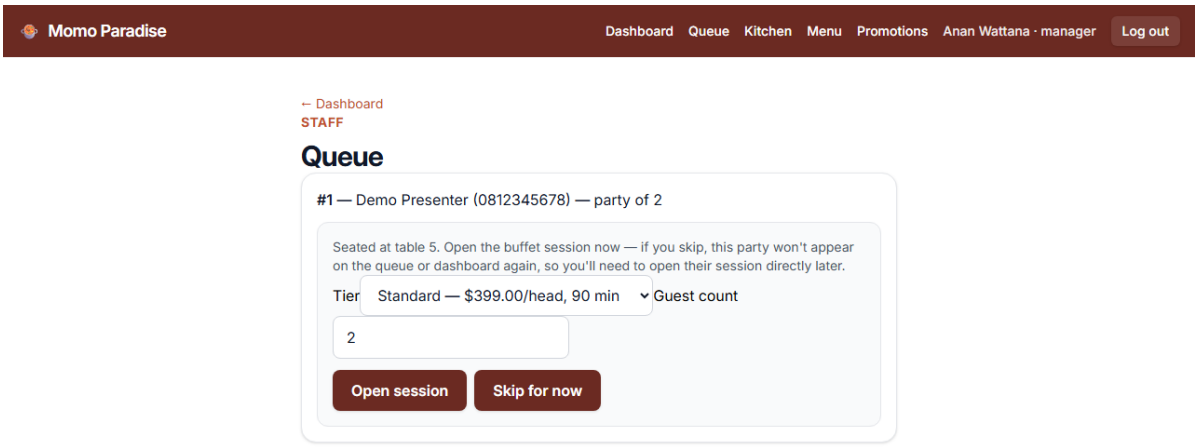


Figure 10: Seating a party: fn_seat_reservation assigns the table, then fn_open_session starts the timed session (tiers listed by fn_get_tiers).

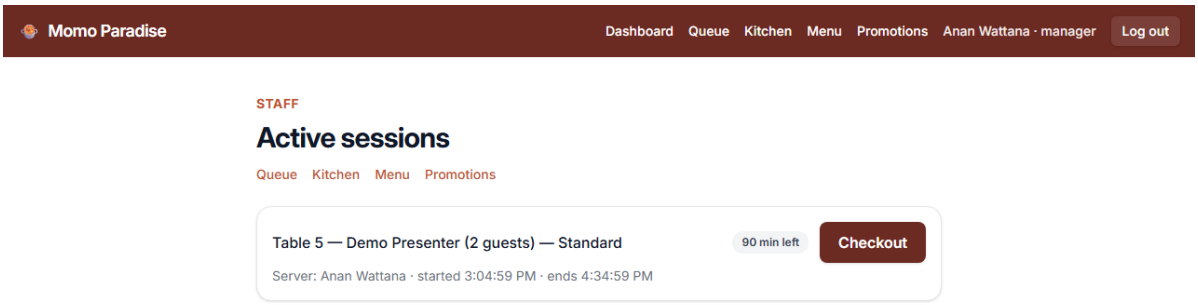


Figure 11: Dashboard: active sessions with elapsed time and a checkout button — powered by fn_get_active_sessions.

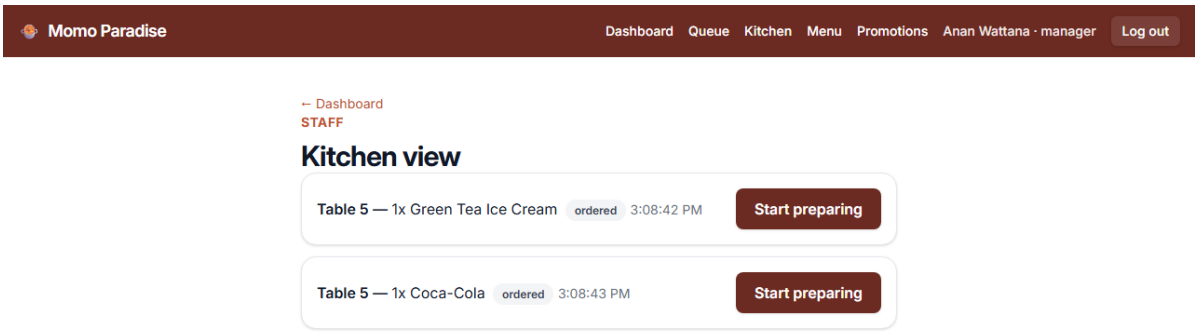


Figure 12: Kitchen view: live unserved orders, oldest first — fn_get_kitchen_orders (indexed) feeds the list; fn_update_order_status advances each dish.

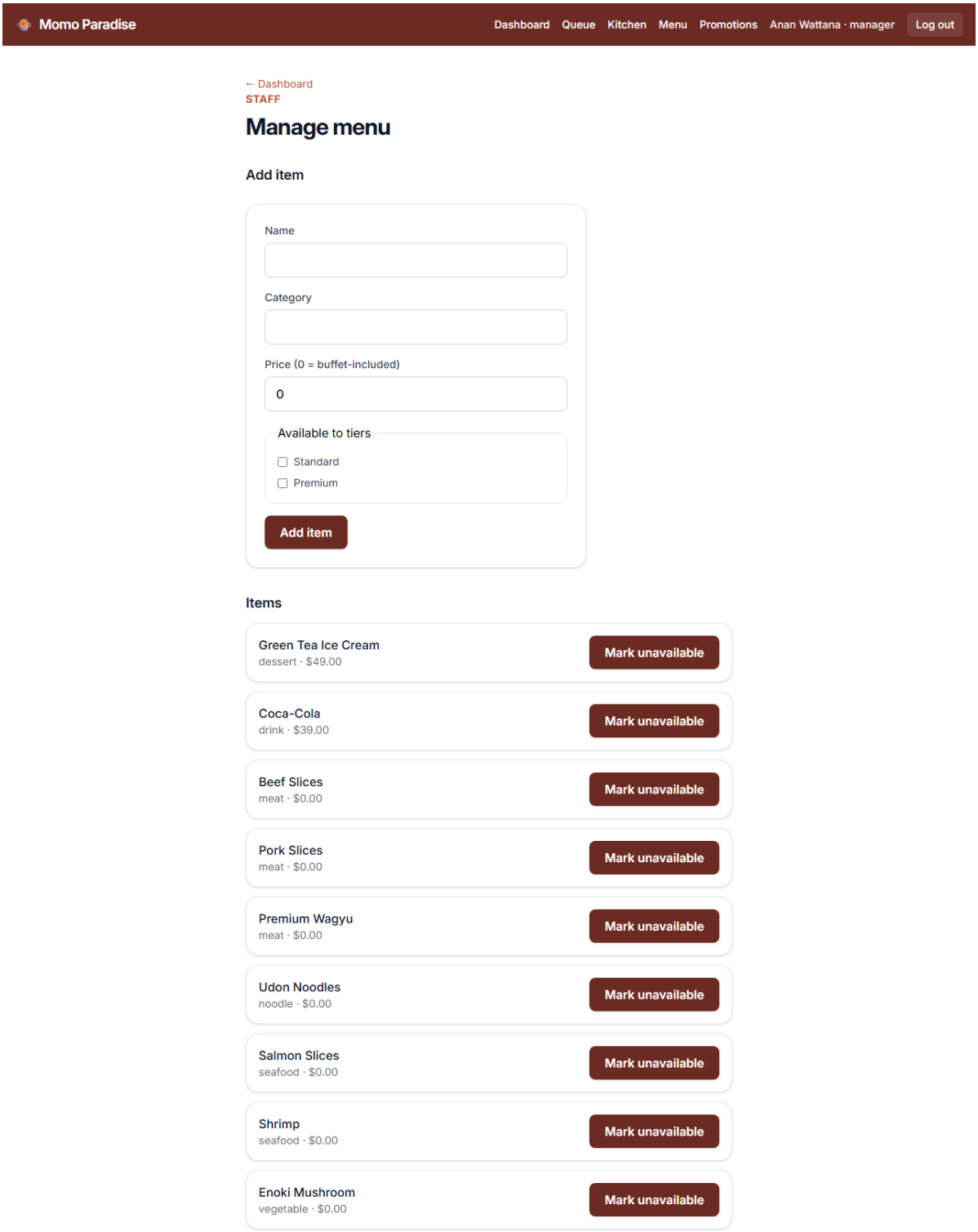


Figure 13: Menu management — fn_get_menu_items lists the full inventory; fn_add_menu_item and fn_update_item_availability modify it.

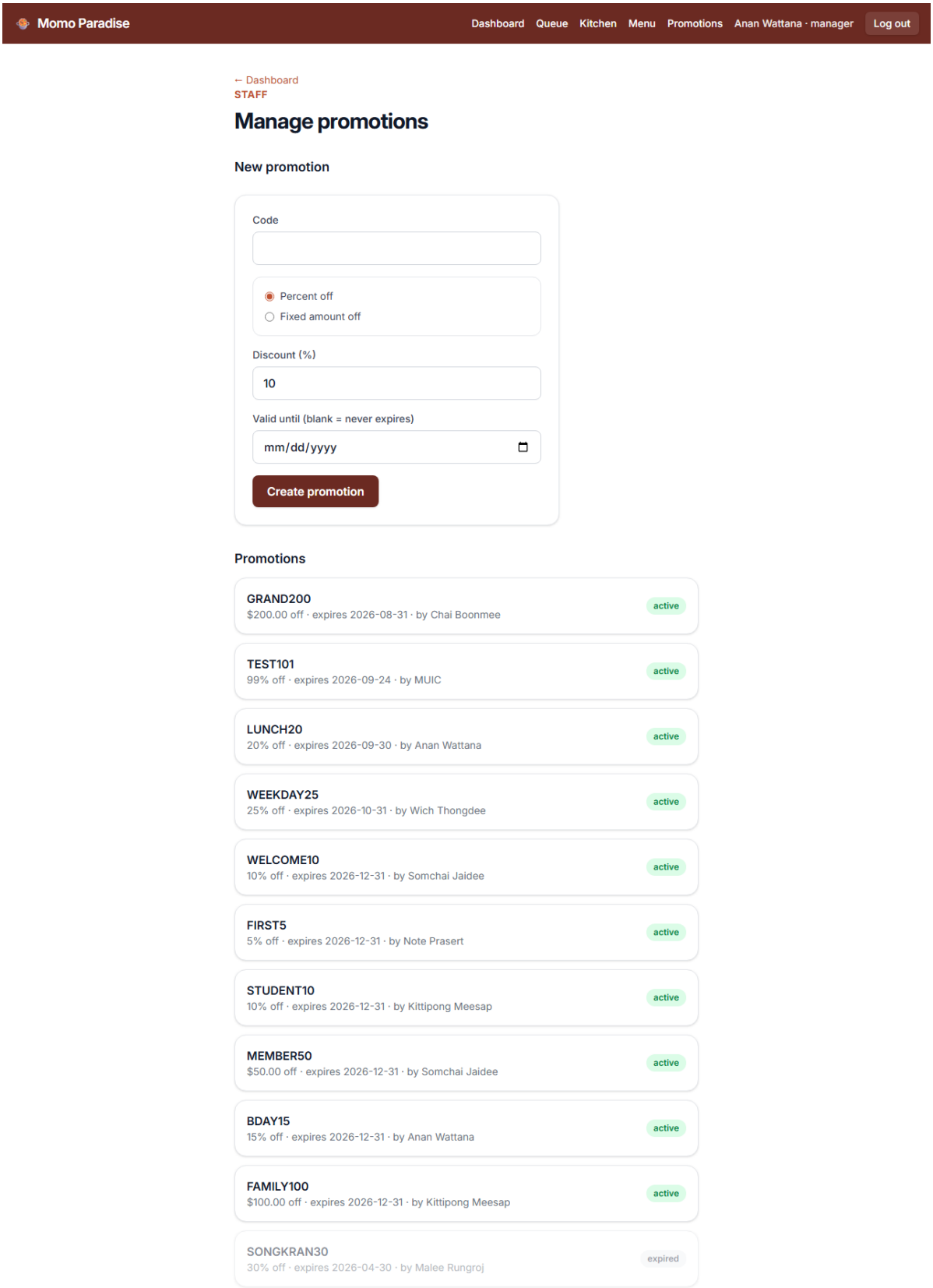

Momo Paradise
Dashboard
Queue
Kitchen
Menu
Promotions
Anan Wattana · manager
Log out
← Dashboard
STAFF
Manage promotions
New promotion
Code
Percent off
Fixed amount off
Discount (%)
10
Valid until (blank = never expires)
mm/dd/yyyy
Create promotion
Promotions
GRAND200
$200.00 off · expires 2026-08-31 · by Chai Boonmee
active
TEST101
99% off · expires 2026-09-24 · by MUIC
active
LUNCH20
20% off · expires 2026-09-30 · by Anan Wattana
active
WEEKDAY25
25% off · expires 2026-10-31 · by Wich Thongdee
active
WELCOME10
10% off · expires 2026-12-31 · by Somchai Jaidee
active
FIRST5
5% off · expires 2026-12-31 · by Note Prasert
active
STUDENT10
10% off · expires 2026-12-31 · by Kittipong Meesap
active
MEMBER50
$50.00 off · expires 2026-12-31 · by Somchai Jaidee
active
BDAY15
15% off · expires 2026-12-31 · by Anan Wattana
active
FAMILY100
$100.00 off · expires 2026-12-31 · by Kittipong Meesap
active
SONGKRAN30
30% off · expires 2026-04-30 · by Malee Rungroj
expired

Figure 14: Promotion management — fn_get_promotions lists codes; fn_create_promotion adds percent or fixed discounts.

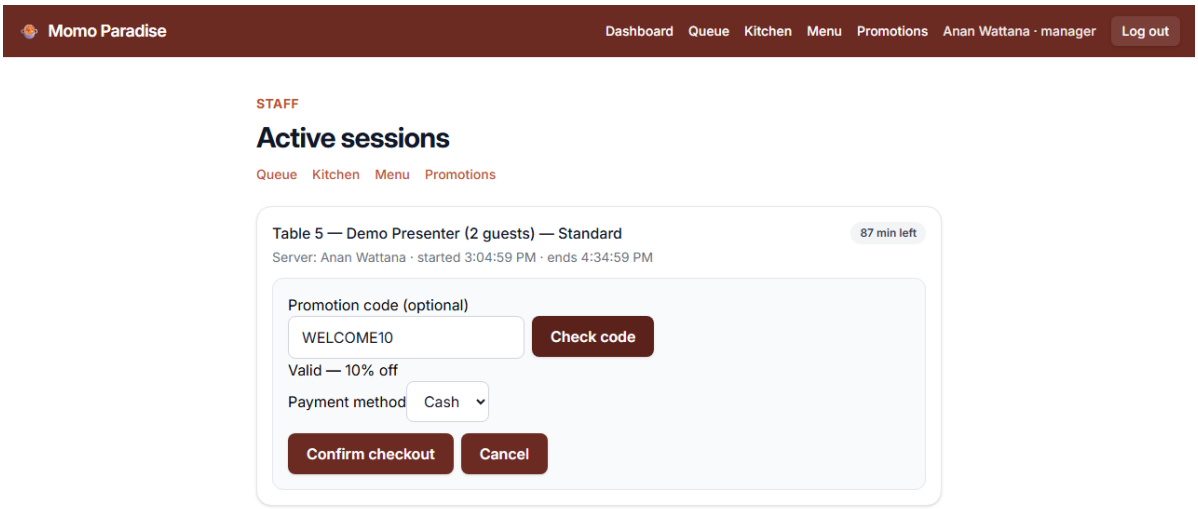


Figure 15: Checkout: fn_validate_promotion checks the code live before confirming.

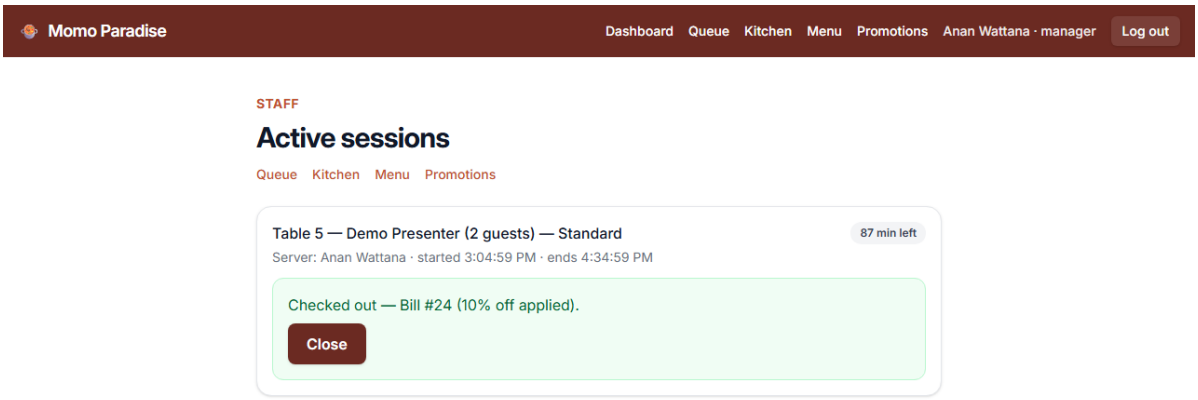


Figure 16: Checkout confirmation — fn_checkout creates the bill, applies the discount, awards points, and frees the table in one transaction; fn_get_bill renders the receipt.

6. Key Functionality: SQL Functions

All 28 functions are organized by screen responsibility — each screen of the app maps to the functions that power it. The application is granted EXECUTE on these functions and nothing else.

Group	Function	What it does
Auth	fn_register_customer	Creates a customer with a bcrypt-hashed password and a membership row (0 points); returns the new customer_id.
	fn_login_customer	Returns the customer row only on a correct email + password hash match; wrong credentials return 0 rows.
	fn_login_staff	Staff login; returns the staff row including role and branch.
	fn_get_session_owner	Authorization helper: lets the backend verify a dining session belongs to the logged-in customer.
Membership	fn_get_membership	Points balance and member tier for a customer.
	fn_get_point_history	Chronological earn/redeem history.
Customer journey	fn_get_branches	All branches, alphabetical — the home-screen picker.

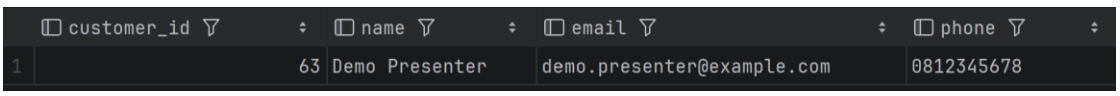
	fn_get_available_tables	Free tables at a branch seating at least the party size, smallest fit first.
	fn_create_reservation	Timed booking (status 'reserved') or, with no slot time, a walk-in queue entry (status 'queued').
	fn_get_tier_menu	Only the dishes the session's tier may order (join through tier_menu_item), available items only.
	fn_place_order	Inserts an order line after checking the tier rule; snapshots the current price into unit_price.
	fn_get_session_orders	All order lines of a session with status.
	fn_get_current_bill	Running total: tier price × guests + Σ(qty × unit_price).
Staff — queue & sessions	fn_get_queue	Waiting parties at a branch, oldest first.
	fn_seat_reservation	Marks a reservation 'seated' and its tables occupied.
	fn_open_session	Opens a timed dining session (walk-in or pre-seated reservation).
	fn_get_active_sessions	Dashboard list of open sessions with elapsed time.
	fn_get_tiers	Buffet tiers of a branch, for the open-session dialog.
Staff — kitchen	fn_get_kitchen_orders	Unserved dishes for active sessions, oldest first (indexed).
	fn_update_order_status	Advances an order line (ordered → preparing → served).
Staff — menu	fn_get_menu_items	A branch's full menu inventory, including unavailable items.
	fn_add_menu_item	Adds a dish and links it to tiers in the same call.
	fn_update_item_availability	Toggles a dish on/off the menu.
Staff — promotions	fn_create_promotion	Creates a coupon code (percent or fixed).
	fn_get_promotions	Lists codes with validity.
	fn_validate_promotion	Checks a code before checkout and returns its discount.
Staff — checkout	fn_checkout	One transaction: creates the bill, applies the discount, awards points, closes the session, frees the tables. Row-locked (FOR UPDATE) against double checkout.
	fn_get_bill	Itemized finished bill for the post-checkout receipt.

6.1 Example Calls and Results

The examples below were run in DataGrip against the live database on Render.

Login — correct credentials return the customer row; anything wrong returns 0 rows, without revealing which field was incorrect:

```
SELECT * FROM fn_login_customer('demo.presenter@example.com', 'password123');
```



	customer_id	name	email	phone
1	63	Demo Presenter	demo.presenter@example.com	0812345678

Figure 17: fn_login_customer returns the customer row on a correct bcrypt match.

Tier-filtered menu — session 31 only sees dishes linked to its tier through tier_menu_item:

```
SELECT * FROM fn_get_tier_menu(31);
```

	item_id	name	category	price	available
1	23	Green Tea Ice Cream	dessert	49	true
2	22	Coca-Cola	drink	39	true
3	16	Beef Slices	meat	0	true

7 rows

Figure 18: fn_get_tier_menu returns only the dishes this session's tier may order.

The tier rule in action — ordering a dish outside the session's tier is rejected by the database itself:

```
SELECT * FROM fn_place_order(4, 3, 1);
-- ERROR: Item 3 is not included in this session's tier
```

Orders of a session — each line snapshots the price it was ordered at:

```
SELECT * FROM fn_get_session_orders(31);
```

	ıe_id	item_name	quantity	unit_price	line_total	status	ordered_at
1	49	Coca-Cola	1	39	39	ordered	2026-07-10 15:08
2	48	Green Tea Ice Cream	1	49	49	served	2026-07-10 15:08

Figure 19: fn_get_session_orders: order lines with quantity, snapshot price, and status.

Checkout — one call, one transaction. A live test produced: buffet 798.00 + extras 88.00 − 10% discount 132.90 = 753.10 final, 7 points earned, session closed, table freed:

```
SELECT * FROM fn_checkout(1, 'WELCOME10', 'credit_card');
```

Point history — the earn from a checkout appears on the customer's profile immediately:

```
SELECT * FROM fn_get_point_history(63);
```

	transaction_id	change_amount	type	bill_id	created_at
1	26	7	earn	24	2026-07-10 15:08:50.931283

Figure 20: fn_get_point_history: the points earned by the bill above.

7. Design Quality, Security, and Efficiency

7.1 Normalization

The schema satisfies Third Normal Form: all values are atomic, the composite-key junction tables (tier_menu_item, reservation_table) carry no non-key attributes, and there are no transitive dependencies. The one deliberate exception is the price snapshot described in Section 4.3: order_line.unit_price and the stored totals in bill duplicate derivable data on purpose, so that historical bills remain correct when menu prices change.

7.2 Security

Security is enforced in the database itself, in three layers:

- **Passwords: bcrypt in the database.** Using the pgcrypto extension, passwords are stored only as crypt(pw, gen_salt('bf')) hashes. No plaintext is ever stored or compared, and a failed login does not reveal which field was wrong.
- **SQL injection: parameterized queries only.** Flask has exactly one database helper, call_fn(), which executes SELECT * FROM fn_x(%s, %s) with bound parameters. There is no string-built SQL anywhere in the application.
- **Least privilege: an EXECUTE-only role.** The application connects as momo_app, a role with ALL privileges REVOKED on every table and sequence and GRANT EXECUTE on the fn_* functions only. The functions are SECURITY DEFINER with a pinned search_path. Even a successful injection could therefore only call the same 28 functions the app already uses.

The lockout is easy to demonstrate live:

```
SET ROLE momo_app;
SELECT * FROM customer;
-- ERROR: permission denied for table customer
```

7.3 Query Efficiency

Indexes were added for the two hottest read patterns, identified from how the app's screens actually poll the database:

- **idx_reservation_branch_time on reservation (branch_id, slot_time)** — powers availability lookups: every customer checking a branch's free tables filters reservations by branch and time window.
- **idx_order_line_session_status on order_line (session_id, status)** — powers the kitchen view, which polls for unserved lines of active sessions, and the running bill, which sums a session's order lines.

Write paths are kept safe under concurrency as well: fn_checkout locks the dining session row with SELECT … FOR UPDATE, so two staff members cannot check out the same session twice.

8. Conclusion and Future Work

The finished platform covers the complete operational loop of a Momo Paradise visit — reserve or queue, seat, dine with tier-enforced ordering, and check out with promotions and points — with every business rule enforced inside PostgreSQL. The application layers stay thin by design: the database is the single source of truth, and the EXECUTE-only role means it defends itself even if a layer above is compromised.

Future work, in rough priority order:

- A recovery screen for parties that were seated but whose session was not opened immediately (currently staff must complete both steps together).
- Public deployment. The system runs on Render's database already; the web tiers were demoed via Docker Compose instead of a public URL to avoid free-tier cold starts during a live presentation.
- Features deliberately left out of scope: gift cards, advertising, and multi-visit analytics.

8.1 Team Contributions

Member	ID	Main responsibilities
Chaiyanun Sakulsaowapakkul	6681299	Database design: schema, SQL functions, security model, deployment scripts
Sorawich Pimsen	6681297	Frontend: React application, UI design, customer and staff screens
Nathanon Chirattithphan	6780756	Backend: Flask API, Docker Compose environment

8.2 Project Repository and Assets

- Source code (schema, functions, backend, frontend, deploy scripts): https://github.com/taro-5453/database-webapp
- Database: PostgreSQL hosted on Render; deployed and verified with script/deploy.sh and script/verify.sh.
- To run the whole app locally: docker compose up --build, then open http://localhost:8080.